

Campaign Technology Suite: handover and deployment

What has been built in Phases 0, 1 and 2, how the code is organised, how to run the demo, and how to deploy it for India and Canada.

6 of 8

tools demo-ready, in one codebase for India and Canada

61

automated tests passing (unit + real-Postgres integration)

3 apps

API with voter page, campaign dashboard, offline booth worker app

Rules first

every call and send passes the compliance engine

01 · SUMMARY

What this handover covers

The Campaign Technology Suite is one codebase with two editions, India (`IN`) and Canada (`CA`). Country differences live in a region config and a compliance engine, not in scattered code. Each campaign is a tenant, isolated at the database level.

6 / 8

tools built and demo-ready

61

automated tests, all passing

~6,000

lines of TypeScript, SQL, HTML

2

seeded demo campaigns (IN + CA)

Not production-ready yet. The demo is safe to show and to host: demo campaigns can never dial a real person. Before any live campaign, the P0 items in section 07 and in the roadmap document must be done, together with the legal reviews.

02 · WHAT IS BUILT

Built in Phases 0, 1 and 2

Area	What it does	Where
Region config	Languages, providers, approval gate (MCMC certificate in IN, owner approval in CA), silence window, calling hours, AI disclosure text, donation rules. Unconfirmed legal values are <code>null</code> and block sends.	<code>packages/regions</code>
Compliance engine	One function, <code>checkOutbound()</code> , runs every gate: approval/certificate, DLT template, silence window, calling hours, consent and opt-outs, donation rule, AI disclosure, campaign named, rate limit, spend. Checked when a run starts and before every single call.	<code>packages/compliance</code>
Database	Postgres with Row-Level Security per campaign, one-race-one-client unique rule, encrypted phone numbers, append-only audit log. 3 migrations.	<code>packages/db</code>
Channels	Voice and SMS behind one interface. Simulated channels for demos; Vapi (voice) and Twilio SMS (CA) for live. Demo campaigns always get simulated channels.	<code>packages/channels</code>
Auth and team	Phone + OTP login, JWT with refresh, roles (candidate, manager, finance agent, coordinator, booth worker, polling agent, office staff).	<code>apps/api</code>
Content and approvals	Scripts, SMS, voter-page blocks, assistant answers, share messages. Scripts must open with the AI disclosure. Any edit resets to draft.	<code>apps/api</code>

Area	What it does	Where
AI calls and surveys	Run builder, per-call compliance, simulated dialler, survey results by area, opt-out handling, callbacks, masked call log, pause/resume, evidence pack. Vapi webhook for live.	<code>apps/api/src/modules/calls</code>
Voter page	<code>/v/:slug</code> : Punjabi/Hindi/English, pick area, hear the plan read aloud, sign up with picture buttons and spoken consent, ask the assistant.	<code>apps/api/src/public/voter.html</code>
AI assistant	Answers only from approved content and names the source; voting logistics go to the official site; otherwise hands off to the team.	<code>modules/assistant</code>
Share links and QR	Per-worker links counting opens and sign-ups; approved message to paste into WhatsApp by hand; printable QR.	<code>modules/share</code>
Booth worker app	<code>/w</code> : installable, works offline, own areas only, last 4 digits of numbers, big result buttons, idempotent sync.	<code>apps/api/src/public/worker</code>
Events and volunteers	Permission gate for IN rallies, sabhas and vehicles; shifts; consented SMS reminders; event costs flow to finance once.	<code>modules/ops</code>
Lawn signs (CA)	Requests, route planning for tonight's drops, placed/collected status.	<code>modules/ops</code>
Finance	Expenditure (IN, district rate list checks) and campaign finance (CA, contributor limits, receipts). Flags, sign-off lock, CSV registers after sign-off.	<code>modules/finance</code>
Dashboard	React app at <code>/admin</code> for all of the above.	<code>apps/dashboard</code>
Demo seed	Two demo campaigns with areas, content, surveys, 220 fake contacts, workers, events, finance and signs.	<code>apps/api/src/scripts/seed-demo.ts</code>

How the repository is organised

```
campaign-suite/
  apps/
    api/           Express API, voter page (/v), worker app (/w), webhooks, demo seed
                  src/modules/ calls, assistant, public (hub), share, field, ops, finance
                  test/      integration tests against real Postgres
    dashboard/     React + Vite admin, built to dist/ and served at /admin
  packages/
    regions/       IN and CA config
    compliance/    the compliance engine + unit tests
    channels/       voice/SMS adapters (mock, Vapi, Twilio)
    db/            SQL migrations (source of truth), Drizzle schema, migrator
    scripts/        guardrails.mjs (CI blocks WhatsApp automation libraries)
  .github/workflows/ci.yml, docker-compose.yml, .env.example, README.md
```

Automated tests

Suite	Tests	What it proves
Compliance engine	22	Every gate for IN and CA, time zones, run vs send scope, spend warnings
Channels	4	Demo always simulated, live fails closed without keys, Vapi plays the approved script first
API integration	30	Login, tenant isolation (RLS), one race one client, approvals, calls end to end, opt-outs, evidence, hub, assistant, share links, team, offline sync, permissions, finance lock and export, CA limits and routes
Finance and route rules	5	Rate-list and limit flags, CSV injection safety, route ordering

Run them with `pnpm guardrails && pnpm typecheck && pnpm test` (integration tests need `TEST_DATABASE_URL` and `TEST_APP_DATABASE_URL`; CI sets these up automatically).

04 · RUN IT LOCALLY

Run the demo on a laptop

- 1 Install Node 22, pnpm 9 and Docker. Unzip the code and run `pnpm install`.
- 2 Copy `.env.example` to `.env`. Generate the two keys with `openssl rand -base64 32` for `PHONE_ENC_KEY` and `PHONE_HASH_KEY`. Set `DEV_RETURN_OTP=true` for the demo.
- 3 Run `pnpm db:up` (Postgres + Redis) and then `pnpm demo`. This builds the dashboard, applies migrations, seeds both demo campaigns and starts the API on port 4000.
- 4 Open `http://localhost:4000/admin` (India `+919999900001`, Canada `+12045550001`), the voter pages `/v/demo-ludhiana` and `/v/demo-winnipeg-ward3`, and the worker app `/w` (India `+919999900002`, Canada `+12045550002`).

Keep `PHONE_ENC_KEY` and `PHONE_HASH_KEY` safe and unchanged. If they change, stored phone numbers can no longer be read or matched. Re-seed the demo if you rotate them.

Demo script that sells

1. Phone: open the voter page, pick an area, press the big listen button, ask the assistant a question and then "where is my booth".
2. Dashboard: Calls → create a run → show the rules check → start → watch survey results fill in by area → download the evidence pack.
3. Phone: open , mark a few houses, switch to airplane mode, mark more, switch back and watch them send.
4. Dashboard: Events → try to confirm the rally without permission (refused), Expenditure → show the flags and the limit meter. Canada: Lawn signs → tonight's route.

The Canada demo follows real calling hours in Winnipeg time. Outside 9:00–21:30 on weekdays (10:00–18:00 at weekends) its runs are blocked; that is the product working.

Deployment

There are two kinds of deployment. A **demo server** can go up now to share a live link with candidates. A **production deployment** per country comes after the P0 work in section 07.

A. Demo server (can be done now)

- 1 One small AWS EC2 instance in `ap-south-1` (Ubuntu, 2 vCPU, 4 GB) is enough. Install Node 22, pnpm, PM2 and Postgres 16 (or use RDS).
- 2 Create the app role with a strong password **before** migrating (the migration only creates `cs_app` if it does not exist):

```
CREATE ROLE cs_app LOGIN PASSWORD '<long random>';
```
- 3 Set env: `NODE_ENV=demo`, `DEV_RETURN_OTP=true`, `PUBLIC_BASE_URL=https://demo.<your-domain>`, database URLs, JWT secrets, phone keys. Leave all provider keys empty.
- 4 `pnpm install --frozen-lockfile && pnpm build:dashboard && pnpm db:migrate && pnpm seed:demo`
- 5 Start with PM2: `pm2 start "pnpm --filter @cs/api start" --name campaign-api` and `pm2 save`. Run **one** instance only (see section 07).
- 6 Put Nginx (or an AWS load balancer) in front with HTTPS (Let's Encrypt). HTTPS is required: the worker app's offline mode only works on HTTPS.

`DEV_RETURN_OTP=true` shows the login code on screen. It is refused when `NODE_ENV=production`. Use it on the demo server only, never with real campaign data.

B. Production, one deployment per country

Item	India edition	Canada edition
AWS region (data residency)	ap-south-1 (Mumbai)	ca-central-1
Database	RDS Postgres 16, Multi-AZ, encrypted, daily snapshots	Same
API + workers	ECS/Fargate or EC2 + PM2, behind an ALB with TLS	Same
Voice	Vapi with a 140-series-capable number (after provider confirmation)	Vapi with a Twilio number
SMS	DLT-registered provider (adapter still to build)	Twilio Messaging Service with STOP handling
Login OTP	SMS OTP provider (still to build)	SMS OTP provider (still to build)
DEPLOY_REGION	IN	CA

- 1 Create the database, then the owner role for migrations and `cs_app` with a strong password; store both in AWS Secrets Manager.

- 2 Build: `pnpm install --frozen-lockfile && pnpm build:dashboard`. Migrate as the owner: `DATABASE_URL=... pnpm db:migrate`. Never run the demo seed in production.
- 3 Configure env from Secrets Manager (table on the next page) with `NODE_ENV=production` and `DEV_RETURN_OTP=false`.
- 4 Point the domain at the load balancer with TLS. The app trusts one proxy hop for client IPs.
- 5 Vapi: import the phone number, create the base assistant, set the server URL to `https://<domain>/webhooks/vapi` with the secret header `x-vapi-secret`, and put the IDs in env.
- 6 Turn on monitoring: error tracking, uptime checks on `/health`, database alarms, and log retention.

Environment variables

Variable	Required	Purpose
APP_DATABASE_URL	yes	Runtime connection as <code>cs_app</code> (Row-Level Security applies)
DATABASE_URL	migrate/seed	Owner connection, only for migrations and the demo seed
JWT_SECRET, JWT_REFRESH_SECRET	yes	Signing keys for access (15 min) and refresh (30 days) tokens
PHONE_ENC_KEY, PHONE_HASH_KEY	yes	32-byte base64 keys for phone encryption and lookup; never rotate without a migration plan
DEPLOY_REGION	yes	IN or CA: which edition new campaigns on this deployment belong to
PUBLIC_BASE_URL	yes	Used in share links and QR codes
NODE_ENV	yes	production on live servers
DEV_RETURN_OTP	demo only	true shows the login code; refused in production
OTP_PROVIDER	yes	Currently only <code>console</code> ; an SMS provider is P0
VAPI_API_KEY, VAPI_PHONE_NUMBER_ID, VAPI_ASSISTANT_ID, VAPI_WEBHOOK_SECRET	live calls	Live voice; demo campaigns ignore them
TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_MESSAGING_SERVICE_SID	CA SMS	Canada SMS
ANTHROPIC_API_KEY, ANTHROPIC_MODEL	optional	Lets the assistant rephrase approved answers (it must still cite a source)
PORT	no	Default 4000

Security and readiness checklist

- ☐ **SMS OTP provider** for login (India via DLT, Canada via Twilio). Today codes only print to the server log.
- ☐ **Move call runs to Redis + BullMQ** and lock queued rows (`FOR UPDATE SKIP LOCKED`). The current in-process runner is safe with one instance only; two instances could call the same person twice.
- ☐ **Redis-backed rate limits** for public endpoints (current limiter is per process).
- ☐ `cs_app` created with a strong secret password; database not publicly reachable; backups tested by restoring.
- ☐ Legal review, India and Canada: calling hours, disclosure wording in each language, consent text versions, retention period, evidence pack format, contracts with the one-race-one-client clause.
- ☐ Vapi request and webhook fields checked against current Vapi docs with a test call to a team phone.

- ☐ India: DLT registration and adapter; MCMC process agreed with the first client.
- ☐ Pre-recorded Punjabi and Hindi audio for the voter page; self-hosted fonts for low-data users.
- ☐ Load test results day and call endpoints; on-call plan for election week.

08 · RULES BUILT INTO THE CODE

Guardrails the team must keep

- No WhatsApp automation: CI fails if a WhatsApp library is added. Share links are copy-only, pasted by people.
- No caste or religion fields: contact endpoints reject unknown fields.
- Electoral roll or imported lists require a source proof file.
- Demo campaigns can never use live channels; live campaigns without provider keys fail closed.
- Calling hours for live campaigns can only be changed by Wayne E Solutions after legal confirmation.
- Any content edit resets approval; certificates cover exact text only.
- Finance periods lock after sign-off; exports only for signed-off periods.